Term: Spring 2026 **Course:** COMP 562
Homework: Project 1 **Name:** Gianna Pedicini
NetID: gp75 **Github Repo:** https://github.com/giannaped8/COMP562.git

1 Jacobian-based Control Projection

1.1 Implementation

For Task 1, I implemented numerical Jacobian computation in `jac.py` and Jacobian-based velocity control in `sim.py`. The Jacobian $J \in \mathbb{R}^{6\times 7}$ was computed using central finite differences. For each joint q_i, I perturbed the joint angle by a small amount Δq in both positive and negative directions and estimated the derivative using

$$J_{v,i} = \frac{p(q_i + \Delta q) - p(q_i - \Delta q)}{2\Delta q},$$

where $p(\cdot)$ is the end-effector position from forward kinematics. The angular part of the Jacobian was obtained by converting the end-effector orientation to rotation matrices and approximating the derivative of the rotation matrix using the same finite difference approach.

In `PandaSim.execute()`, the Jacobian was recomputed at every simulation step using the current joint configuration. The desired Cartesian twist $v_x = [\dot{x}, \dot{y}, 0, 0, 0, \dot{\theta}]^T$ was projected into joint velocities using the Moore–Penrose pseudoinverse: $\dot{q} = J^+ v_x$. These joint velocities were then applied through velocity control, allowing the end-effector to follow the desired trajectory.

1.2 Experiments

To study the effect of the finite-difference step size Δq, I ran experiments with several different values. The assignment specifies that the average tracking error must satisfy

$$\text{position error} < 0.005 \text{ m}, \qquad \text{orientation error} < 0.017 \text{ rad}.$$

The results of these experiments are shown below:

Δq	Avg Position Error (m)	Avg Orientation Error (rad)	Pass/Fail
10^{-1}	0.000409	0.010509	Pass
10^{-2}	0.000512	0.008229	Pass
10^{-3}	0.000513	0.008238	Pass
10^{-4}	0.000514	0.008758	Pass
10^{-5}	0.000491	0.008831	Pass
10^{-6}	0.000510	0.010881	Pass
10^{-7}	0.162003	1.280528	Fail

1.3 Discussion

The experiments show that the Jacobian-based controller performs well for a wide range of step sizes between 10^{-1} and 10^{-6}. However, when the finite-difference step becomes too small (such as $\Delta q = 10^{-7}$), the approximation becomes dominated by numerical precision errors. In floating-point arithmetic, the difference between the forward kinematics evaluations $f(q + \Delta q)$ and $f(q - \Delta q)$ becomes extremely small and may be close to machine precision. When this small difference is divided by $2\Delta q$, numerical noise is amplified, leading to an unstable estimate of the Jacobian. This particularly affects the rotational component of the Jacobian, which relies on differences between rotation matrices. As a result, the controller computes inaccurate joint velocities and the tracking error becomes very large, causing the control to fail the required thresholds.

Overall, the best-performing values were around 10^{-2} and 10^{-3}, which produced the lowest orientation errors while remaining numerically stable. Based on these results, I chose

$$\Delta q = 10^{-3}$$

for the final implementation. This value provides a good balance between numerical stability and an accurate local derivative approximation, and it produced average tracking errors of approximately 0.000513 m in position and 0.008238 rad in orientation, which comfortably satisfy the assignment requirements.

2 Kinodynamic Motion Planning

2.1 Implementation

For Task 2, I implemented two state-checking functions in `pdef.py`: `is_state_valid()` and `is_state_high_quality()`. The validity checker ensures that the robot state satisfies the planning bounds, avoids collisions with the ground, and keeps the end-effector inside the allowed workspace region.

The high-quality checker evaluates the manipulability of the robot using the Jacobian matrix

$$m(J) = \sqrt{\max(0, \det(JJ^\top))},$$

and requires the manipulability to be greater than 0.01. This prevents the robot from entering configurations close to singularities where Jacobian-based control becomes unreliable.

2.2 Discussion

These two checkers affect robot execution in different ways inside `PandaSim.execute()`. The high-quality checker is evaluated at every simulation step immediately after the Jacobian is computed. If `is_state_high_quality(J)` returns false, execution stops immediately. This prevents the robot from continuing into configurations with very low manipulability, where the Jacobian pseudoinverse used for velocity control could generate unstable or excessively large joint velocities. In practice, this acts as a safeguard that keeps the controller operating in well-conditioned regions of the configuration space.

In contrast, the validity checker is evaluated less frequently, approximately every 0.2 seconds (every 12 simulation steps). This check verifies that the current state remains within the allowed state bounds, does not result in collisions with the environment, and keeps the end-effector inside the defined workspace region. If any of these conditions are violated, the execution terminates and the motion is marked invalid. This prevents the planner from exploring trajectories that are physically infeasible or unsafe.

The difference in evaluation frequency reflects the roles of the two checks. The validity check involves more expensive operations such as collision detection and full state verification, so it is performed periodically rather than every step. The high-quality check only depends on the Jacobian, which is already computed for control, allowing it to be evaluated at every step with minimal overhead. Together, these checks ensure both physical feasibility and numerical stability during motion execution: `is_state_valid()` enforces safety constraints, while `is_state_high_quality()` maintains reliable Jacobian-based control.

3 Grasping in Clutter

3.1 Implementation

For Task 3, I implemented the grasp goal test in `GraspGoal.is_satisfied()` so that the Kinodynamic RRT planner from Task 2 can plan toward a feasible grasp configuration. The robot joint angles are used to compute the end-effector pose via forward kinematics, and the end-effector position is converted to the world frame using the fixed Panda base offset.

The relative position between the end-effector and the target cube is then used to compute two geometric distances: the perpendicular distance from the cube center to the gripper plane (d_1) and the distance along the gripper opening direction (d_2). The orientation alignment between the end-effector and the cube is measured by the wrapped angle difference γ, considering the four equivalent orientations of the square cube.

The grasp goal is satisfied when

$$d_1 < 0.01, \qquad d_2 < 0.02, \qquad \gamma < 0.2.$$

These conditions ensure that the cube lies within the gripper opening and that the gripper orientation is properly aligned for grasping.

4 Trajectory Optimization

In this task, trajectory optimization is applied to improve the motion plans generated by the Kinodynamic Rapidly-Exploring Random Tree (Kinodynamic RRT) planner implemented in Task 2. Sampling-based planners such as RRT are effective at quickly finding feasible solutions in high-dimensional spaces, but the resulting trajectories are often inefficient and contain unnecessary control segments. Therefore, a post-processing optimization step is introduced to refine the trajectory and improve execution efficiency.

Following the project description, the solution trajectory produced by the planner is used as an initial solution and then refined using stochastic trajectory optimization techniques. The goal is to reduce trajectory cost while maintaining task feasibility and collision-free behavior.

For this project, optimization was performed on the **relocation task**, where the robot must push the target cube into the specified goal region on the table.

4.1 Implementation

The trajectory optimization algorithm is implemented in `opt.py`. The planner first generates an initial feasible trajectory using the Kinodynamic RRT algorithm from Task 2. This trajectory consists of a sequence of controls, where each control is represented as `[x_dot, y_dot, θ_dot, d]`. corresponding to the end-effector linear velocities in the x and y directions, angular velocity about the z axis, and the duration of the control.

The optimizer improves this trajectory by iteratively modifying the control sequence. At each iteration, a candidate trajectory is generated by randomly applying one of the following stochastic modification operations:

- **Control removal:** randomly removes a control segment to simplify the trajectory.
- **Duration shrinking:** reduces the duration of a control segment to shorten the total motion time.
- **Control perturbation:** adds Gaussian noise to the control parameters to explore nearby trajectories.

Each candidate control sequence is executed using the simulator propagation function. If the resulting trajectory remains valid and still satisfies the goal condition, its cost is evaluated. The candidate replaces the current solution only if it achieves a lower cost. Trajectory quality is measured using a weighted cost function that captures several aspects of motion efficiency:

$$C = C_{duration} + 2C_{path} + 5C_{clutter} \qquad \text{, where}$$

- $C_{duration}$ is the total duration of all control inputs,
- C_{path} is the end-effector path length,
- $C_{clutter}$ is the total displacement of non-target objects.

This formulation encourages shorter trajectories, smoother end-effector motions, and minimal disturbance to surrounding objects. The optimization therefore acts as a local refinement step that improves the efficiency of trajectories generated by the sampling-based Kinodynamic RRT planner.

4.2 Experiments

To evaluate the effectiveness of the trajectory optimizer, experiments were conducted using multiple random seeds and different optimization iteration budgets.

The following parameters were used:

- Random seeds: $0, 1, 2, 3, 4, 5$
- Optimization iterations: $0, 100, 300$
- Planning time budget: 120 seconds

For each run, the following metrics were recorded:

- Initial trajectory cost
- Optimized trajectory cost
- Number of control segments before and after optimization
- Percentage cost improvement
- Total runtime

The full experimental results across all seeds and iteration counts are shown in Figure 1.

Task	Seed	Iterations	Solved?	Runtime (s)	Initial Controls	Optimized Controls	Initial Cost	Optimized Cost	Cost Improvement %
Relocation	0	0	TRUE	16.6030941	6	6	5.783894378	5.783894378	0
Relocation	0	100	TRUE	64.78357697	7	7	8.892152436	8.203944386	7.739499017
Relocation	0	300	TRUE	91.08976889	5	5	7.064662629	5.637940948	20.19518491
Relocation	1	0	TRUE	22.68874598	9	9	9.090272052	9.090272052	0
Relocation	1	100	TRUE	58.2647481	4	4	6.711030889	6.632133801	1.175632906
Relocation	1	300	TRUE	114.9450402	6	6	7.11301714	5.434369404	23.59965825
Relocation	2	0	TRUE	13.54324508	7	7	7.853228755	7.853228755	0
Relocation	2	100	TRUE	68.99822116	8	7	7.272913788	6.456134844	11.23042248
Relocation	2	300	TRUE	96.51080585	5	5	6.690897897	6.07003267	9.279251252
Relocation	3	0	TRUE	23.0666039	9	9	8.690236585	8.690236585	0
Relocation	3	100	TRUE	77.59541726	8	7	9.796340948	9.083887145	7.272652176
Relocation	3	300	TRUE	145.219028	9	7	9.812006392	6.155167771	37.26901997
Relocation	4	0	TRUE	27.88461518	7	7	9.519459973	9.519459973	0
Relocation	4	100	TRUE	40.76465607	6	6	7.205074998	5.3974892	25.08767498
Relocation	4	300	TRUE	113.2643838	6	6	7.367249455	6.500574783	11.76388389
Relocation	5	0	TRUE	24.59540796	8	8	7.21857232	7.21857232	0
Relocation	5	100	TRUE	73.22312284	8	8	8.606369305	7.603165299	11.65652984
Relocation	5	300	TRUE	150.307663	9	8	8.913882305	6.568215816	26.31475723

Figure 1: Experimental results showing runtime, trajectory costs, and control counts for different seeds and optimization iterations.

Figure 2 illustrates how the percentage cost improvement increases as the number of optimization iterations grows.

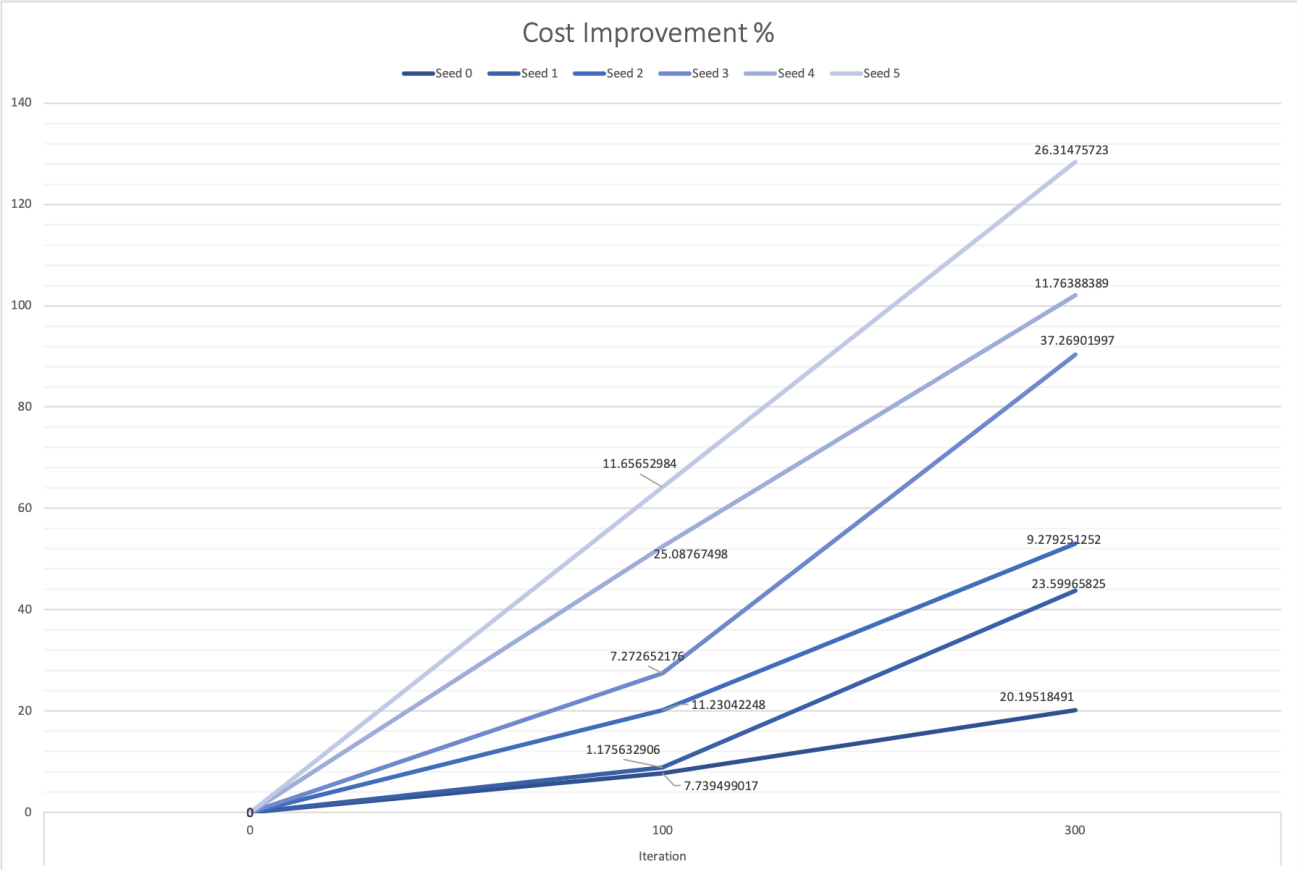

Figure 2: Trajectory cost improvement across different optimization iteration counts.

Across the six seeds tested, trajectory optimization consistently improved the solution quality. With 100 optimization iterations, cost improvements ranged from approximately 1% to 25%. When the optimization budget was increased to 300 iterations, improvements of up to **37.3%** were observed. In many cases, the optimizer also reduced the number of control segments required to execute the trajectory, simplifying the manipulation motion.

Figure 3 shows the initial trajectory produced by Kinodynamic RRT, and Figure 4 shows a comparison between an initial trajectory produced by Kinodynamic RRT and the optimized trajectory after 300 iterations. The red line indicates the end-effector trajectory during execution. Within my experiments, I did not change the trajectory colors between initial and optimized, so viewing Figures 3 and 4 together help protray the difference in the two trajectories and the optimization computed upon the initial path. The optimized trajectory is visibly smoother and often requires fewer control segments, resulting in a more efficient manipulation motion.

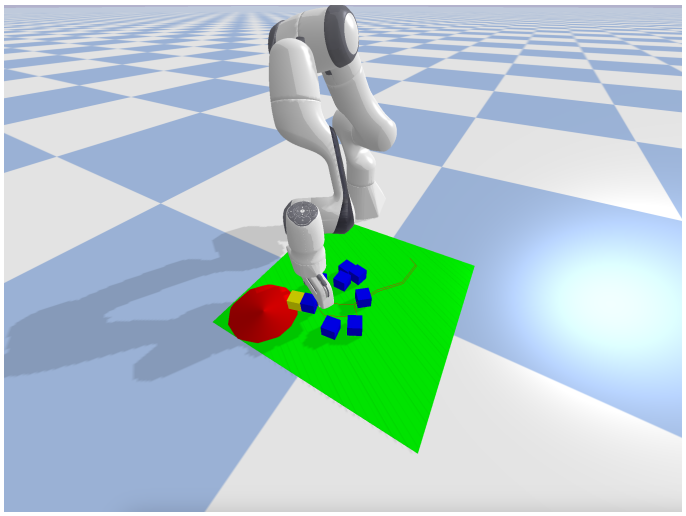

Figure 3: Initial trajectory generated by Kinodynamic RRT.

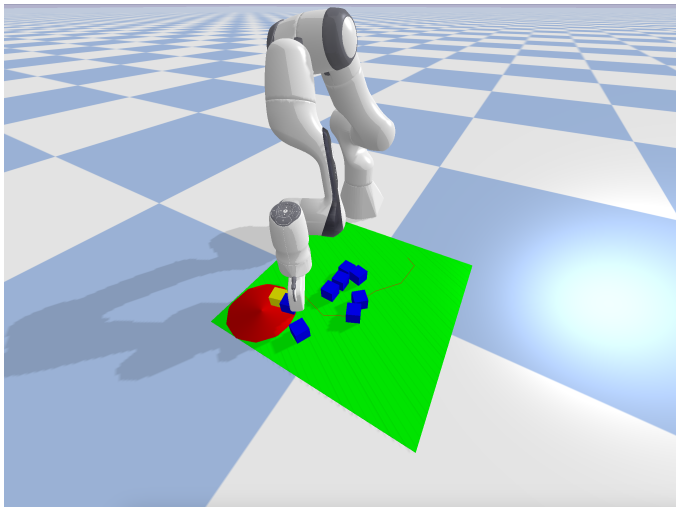

Figure 4: Optimized trajectory after 300 iterations of trajectory optimization.

4.3 Discussion

The results demonstrate that trajectory optimization can significantly improve the quality of motion plans generated by sampling-based planners. While Kinodynamic RRT efficiently finds feasible solutions, the resulting trajectories often contain redundant motion segments and inefficient control durations.

The stochastic optimization process refines these trajectories by exploring local variations in the control sequence and accepting modifications that reduce the overall trajectory cost. Increasing the number of optimization iterations generally improves solution quality, although the rate of improvement decreases as the optimizer approaches a locally optimal trajectory.

Overall, the optimization framework successfully produces shorter and smoother trajectories while minimizing unnecessary motion of surrounding objects. These results highlight the benefit of combining sampling-based planning methods with trajectory optimization to produce efficient manipulation strategies.